

Modulo 7 - Ottimizzazione e debugging

Debugging, performance, build e pubblicazione (Expo-first, con note CLI)

Focus del modulo:

- 1) Debugging e ispezione (Flipper, React Native Debugger, log, network, errori)
- 2) Performance (liste, memoization, lazy loading, immagini, render)
- 3) Build Android/iOS con Expo (EAS) e confronto rapido con React Native CLI
- 4) Test finali e pubblicazione su store (solo aspetti free / requisiti di base)

Obiettivi del modulo

Cosa saprai fare

- Diagnosticare problemi di UI, stato, rete e performance con strumenti pratici.
- Riconoscere i colli di bottiglia più comuni in React Native.
- Ottimizzare liste, rendering e caricamento di schermate.
- Preparare build Android/iOS con flusso Expo (EAS) e capire la differenza col flusso CLI.
- Fare una checklist di test prima della pubblicazione sugli store.

Mappa del modulo

Percorso delle 4 ore

- Blocco A: debugging e strumenti (log, network, error boundaries, ispezione).
- Blocco B: performance optimization (liste, memo, immagini, lazy loading).
- Pausa 15 minuti.
- Blocco C: build Android/iOS (Expo EAS) + confronto rapido CLI.
- Blocco D: test finali e pubblicazione (Google Play / App Store - panoramica base).

1) Debugging e ispezione

Expo-first, con strumenti compatibili RN

Che cosa significa 'debuggare' bene

Approccio

- Prima di cercare la soluzione: definisci sintomo, dove accade, quanto spesso accade.
- Separa problemi di UI, stato, rete, performance, permessi, build.
- Aggiungi log utili (non rumorosi) e riproduci il bug con passi chiari.

Debugging in Expo vs RN CLI

Differenze pratiche

- Expo: flusso rapido con terminale, DevTools, logs, error overlay, emulator/device.
- RN CLI: puoi integrare piu facilmente Flipper completo e tooling nativo.
- Molti concetti sono identici: log, network, stato, performance tracing.

Strumenti principali (modulo)

Panoramica

- Console log e warning (base ma indispensabili).
- Error overlay / stack trace.
- React Native Debugger (Redux, network, console).
- Flipper (piu comune in setup bare/CLI, utile da conoscere).
- Profiler React DevTools (quando disponibile) e misure manuali.

Console log: cosa loggare davvero

Buone pratiche

- Logga input, output e stato prima/dopo una azione importante.
- Usa prefissi chiari: [Auth], [API], [Cart], [Nav].
- Evita di loggare oggetti enormi ad ogni render.
- Rimuovi o riduci i log di debug prima della build finale.

Console log strutturati

Esempio pratico

Utility log

```
export function logStep(scope: string, message: string, data?: unknown) {
  const time = new Date().toISOString();
  if (data !== undefined) {
    console.log(`[${time}] [${scope}] ${message}`, data);
    return;
  }
  console.log(`[${time}] [${scope}] ${message}`);
}

// Uso
logStep("API", "Inizio fetch utenti");
logStep("API", "Risposta utenti", { count: 24 });
logStep("NAV", "Apertura schermata dettaglio", { userId: "u_123" });
```

Errori JS vs errori nativi

Distinzione

- Errori JS: exception, undefined, promise reject, problemi di render.
- Errori nativi: crash build, permessi mancanti, SDK non configurati, moduli non trovati.
- Con Expo, molti errori nativi sono nascosti finche non fai prebuild/build.

Come leggere uno stack trace

Regola pratica

- Parti dalla prima riga del tuo codice (non dal rumore delle librerie).
- Cerca file + riga + funzione chiamata.
- Capisci il contesto: render, evento, useEffect, chiamata API.

Try/catch e gestione errori

Pattern base per API

Pattern async error

```
async function loadUsers() {  
  try {  
    setLoading(true);  
    setError(null);  
  
    const res = await fetch("https://jsonplaceholder.typicode.com/users");  
    if (!res.ok) {  
      throw new Error(`HTTP ${res.status}`);  
    }  
  
    const data = await res.json();  
    setUsers(data);  
  } catch (err) {  
    console.error("[Users] loadUsers error", err);  
    setError("Impossibile caricare gli utenti");  
  } finally {  
    setLoading(false);  
  }  
}
```

Unhandled Promise Rejection

Perche succede

- Hai una funzione async chiamata senza await e senza catch.
- Dentro una Promise lanci un errore ma non lo intercetti.
- Soluzione: await sempre oppure .catch() esplicito.

Pattern corretto per Promise

Await o catch

Promise handling

```
// OK: await + try/catch
async function onPressReload() {
  try {
    await loadUsers();
  } catch (e) {
    console.error(e);
  }
}

// OK: promise + catch
loadUsers().catch((e) => {
  console.error("[Users] onPressReload", e);
});
```

React Native Debugger

Quando usarlo

- Per vedere log, network e stato Redux in un'unica finestra (se compatibile col setup).
- Molto utile quando vuoi spiegare cosa succede in una chiamata API.
- Non è obbligatorio per Expo, ma è comodo da conoscere.

Flipper

Cosa fa e quando serve

- Strumento desktop per ispezionare log, network, layout e plugin vari.
- Spesso piu naturale in React Native CLI (bare) con app nativa avviata localmente.
- In Expo managed non e sempre il flusso principale, ma il concetto resta utile.

Debug del network

Cosa controllare

- URL corretto, metodo HTTP, headers, body, status code.
- Tempo di risposta (lentezza vs errore).
- Parsing JSON e struttura dati attesa.
- Gestione timeout e offline.

Wrapper fetch con timeout

Per debugging e affidabilità

fetchWithTimeout

```
function fetchWithTimeout(url: string, options: RequestInit = {}, ms = 8000) {  
  const controller = new AbortController();  
  const timeout = setTimeout(() => controller.abort(), ms);  
  
  return fetch(url, { ...options, signal: controller.signal })  
    .finally(() => clearTimeout(timeout));  
}  
  
async function getPosts() {  
  const res = await fetchWithTimeout(  
    "https://jsonplaceholder.typicode.com/posts?_limit=10"  
  );  
  
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  
  return res.json();  
}
```

Error Boundary

Per schermate piu robuste

- Intercetta errori durante il render dei componenti figli.
- Mostra una schermata di fallback invece di bloccare tutta la UI.
- Ottimo in app reali per sezioni critiche (dashboard, feed, checkout).

Error Boundary (class component)

React richiede ancora una classe

ErrorBoundary - parte 1/2

```
import React from "react";
import { View, Text, Pressable } from "react-native";

type Props = { children: React.ReactNode };
type State = { hasError: boolean };

export class ScreenErrorBoundary extends React.Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: unknown) {
    console.error("[Boundary] errore schermata", error);
  }

  render() {
    if (this.state.hasError) {
      return (
        <View style={{ padding: 16 }}>
          <Text>Qualcosa e andato storto.</Text>
          <Pressable onPress={() => this.setState({ hasError: false })}>
            <Text>Riprova</Text>
          </Pressable>
        </View>
      );
    }
    return this.props.children;
  }
}
```

Error Boundary (class component)

React richiede ancora una classe (parte 2/2)

ErrorBoundary - parte 2/2

```
        </Text>
      </View>
    );
  }
  return this.props.children;
}
```

Misurare tempi manualmente

Tecnica semplice

- Usa `performance.now()` prima e dopo un'operazione.
- Misura rendering pesanti, parsing, mapping, filtri.
- Confronta prima/dopo una ottimizzazione.

Timer semplice

Misure manuali

Misure performance

```
function measure<T>(label: string, fn: () => T): T {  
  const start = performance.now();  
  const out = fn();  
  const end = performance.now();  
  console.log(`[Perf] ${label}: ${(end - start).toFixed(2)} ms`);  
  return out;  
}  
  
const visibleItems = measure("filter items", () =>  
  items.filter((x) => x.active).slice(0, 100)  
);
```

2) Performance optimization

Render, liste, immagini, lazy loading

Dove nasce la lentezza

Cause comuni

- Liste grandi renderizzate tutte insieme.
- Componenti che ri-renderizzano inutilmente.
- Funzioni e oggetti ricreati ad ogni render.
- Immagini pesanti e non ottimizzate.
- Chiamate API duplicate o sincronizzazione non gestita.

VirtualizedList e FlatList

Perche sono fondamentali

- FlatList renderizza solo una parte della lista (virtualizzazione).
- ScrollView renderizza tutto: bene per contenuti piccoli, male per liste lunghe.
- Per elenchi dinamici usa FlatList quasi sempre.

FlatList: props piu importanti

Ottimizzazione pratica

- keyExtractor stabile.
- renderItem memoizzato o leggero.
- initialNumToRender e maxToRenderPerBatch.
- windowSize per regolare quante celle tenere in memoria.
- getItemLayout se gli elementi hanno altezza fissa.

FlatList ottimizzata

Esempio

FlatList base - parte 1/2

```
import { FlatList, Text, View } from "react-native";
import { memo, useCallback } from "react";

const Row = memo(function Row({ title }: { title: string }) {
  return (
    <View style={{ padding: 12, borderBottomWidth: 1, borderColor: "#e5e7eb" }}>
      <Text>{title}</Text>
    </View>
  );
});

export function PostList({ data }: { data: { id: number; title: string }[] })
{
  const renderItem = useCallback(
    ({ item }: { item: { id: number; title: string } }) => <Row
      title={item.title} />,
    []
  );

  return (
    <FlatList
```

FlatList ottimizzata

Esempio (parte 2/2)

FlatList base - parte 2/2

```
data={data}  
keyExtractor={(item) =>  
String(item.id)}  
renderItem={renderItem}  
initialNumToRender={8}  
maxToRenderPerBatch={8}  
windowSize={7}  
  
    />  
); }
```

getItemLayout (altezza fissa)

Riduce calcoli nello scroll

`getItemLayout`

```
const ROW_HEIGHT = 56;

<FlatList
  data={data}
  keyExtractor={(item) => item.id}
  renderItem={renderItem}
  getItemLayout={(_, index) => ({
    length: ROW_HEIGHT,
    offset: ROW_HEIGHT * index,
    index,
  })}
/>
```

Memoization: quando serve davvero

Regola pratica

- Usa `React.memo` su componenti ripetuti (liste, card) che ricevono props stabili.
- Usa `useMemo` per calcoli costosi (filtri, raggruppamenti).
- Usa `useCallback` quando passi callback a figli memoizzati.
- Non memoizzare tutto: aumenta complessità senza beneficio.

useMemo per filtro pesante

Esempio

useMemo

```
import { useMemo, useState } from "react";

const filtered = useMemo(() => {
  return items
    .filter((item) => item.category === selectedCategory)
    .filter((item) => item.price <= maxPrice)
    .sort((a, b) => a.name.localeCompare(b.name));
}, [items, selectedCategory, maxPrice]);
```


useCallback + child memo

Esempio

useCallback + memo

```
const ProductRow = React.memo(function ProductRow({
  name,
  onSelect,
}): {
  name: string;
  onSelect: () => void;
}) {
  return <Pressable onPress={onSelect}><Text>{name}</Text></Pressable>;
});

const renderItem = useCallback(
  ({ item }) => (
    <ProductRow
      name={item.name}
      onSelect={() => onOpenDetails(item.id)}
    />
  ),
  [onOpenDetails]
);
```

Attenzione a un anti-pattern comune

Inline object e style

- Oggetti creati inline cambiano reference ad ogni render.
- Questo puo invalidare memoization di componenti figli.
- Usa StyleSheet o useMemo per oggetti complessi.

StyleSheet vs inline

Differenza pratica

Style stability

```
// Meno ottimale (nuovo oggetto ad ogni render)
<View style={{ padding: 12, backgroundColor: selected ? "#eef2ff" : "white" }}
/>

// Meglio (stili stabili)
const styles = StyleSheet.create({
  row: { padding: 12, backgroundColor: "white" },
  rowSelected: { backgroundColor: "#eef2ff" },
});

<View style={[styles.row, selected && styles.rowSelected]} />
```

Lazy loading di schermate

Navigazione

- Carica schermate pesanti solo quando servono.
- Riduce tempo di avvio e memoria iniziale.
- Molto utile con navigazione e moduli separati.

React.lazy (concetto) e split

Nota: dipende dal setup

Lazy screen

```
import React, { Suspense } from "react";

const SettingsScreen = React.lazy(() => import("../SettingsScreen"));

export function SettingsEntry() {
  return (
    <Suspense fallback={<Text>Caricamento...</Text>}>
      <SettingsScreen />
    </Suspense>
  );
}
```

Immagini e performance

Checklist

- Riduci dimensioni (non caricare immagini enormi se mostri thumbnail).
- Usa caching (expo-image è utile).
- Placeholder per evitare 'salti' di layout.
- Evita troppe immagini in viewport nello stesso momento.

expo-image (caching)

Esempio

expo-image

```
import { Image } from "expo-image";

<Image
  source={{ uri: product.image }}
  style={{ width: 96, height: 96, borderRadius: 12 }}
  contentFit="cover"
  transition={150}
/>
```

Re-render tree: come spiegarlo

Modello mentale

- Uno state update in un componente avvia un nuovo render di quel componente.
- I figli vengono valutati di nuovo, a meno di memoization efficace.
- Se passi nuove props/reference, i figli memoizzati possono comunque ri-renderizzare.

Come individuare re-render inutili

Tecniche

- Inserisci log nei componenti chiave ('render Row', id).
- Prova React.memo e verifica se i render diminuiscono.
- Controlla callback e oggetti inline.

Log di render (debug temporaneo)

Esempio

Render log

```
const Row = React.memo(function Row({ item }) {  
  console.log("[Render] Row", item.id);  
  return <Text>{item.title}</Text>;  
});
```

Pausa

15 minuti

Riprendiamo tra 15 minuti

3) Build Android e iOS

Expo EAS prima, CLI in confronto

Build in Expo: visione pratica

Flusso base

- Sviluppi localmente con Expo.
- Configuri app.json / app.config.ts (nome, icone, package id, versioni).
- Usi EAS Build per generare build Android/iOS.
- Testi la build reale su device.

Prerequisiti Expo/EAS

Cosa preparare

- Account Expo (gratuito per iniziare).
- Node e progetto Expo funzionante.
- eas-cli installato.
- Identificativi app (Android package / iOS bundle identifier).

Installazione EAS CLI

Comandi

Install EAS

```
npm install -g eas-cli  
eas login  
eas --version
```

Configurazione EAS

Comando iniziale

- eas build:configure crea eas.json e prepara il progetto.
- Puoi definire profili: development, preview, production.
- Per il corso basta capire sviluppo vs produzione.

eas.json (base)

Esempio profili

eas.json

```
{
  "cli": {
    "version": ">= 12.0.0"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  }
}
```

Android build (Expo)

Cosa ottieni

- APK (comodo per test/installazione diretta) oppure AAB (per Play Store).
- Per test interni usa spesso preview/internal.
- Per pubblicazione Play Store userai AAB.

Build Android con EAS

Comandi

EAS Android

```
# Build di test interna  
eas build --platform android --profile preview  
  
# Build produzione (Play Store)  
eas build --platform android --profile production
```

iOS build (Expo)

Nota pratica

- Per iOS reale serve device Apple e provisioning/signing.
- EAS puo guidare la gestione certificati/profili.
- Per mostrare il flusso in corso basta spiegare i passaggi e fare una build demo se possibile.

Build iOS con EAS

Comando

EAS iOS

```
# Build produzione / test  
eas build --platform ios --profile production
```

RN CLI: confronto rapido

Cosa cambia

- Build locali con Android Studio / Gradle e Xcode.
- Piu controllo, ma piu setup (SDK, signing, pods, variabili).

RN CLI - comandi tipici (rapido)

Solo confronto

CLI quick compare

```
# Android (bare)
npx react-native run-android
```

```
# iOS (bare, macOS)
npx pod-install
npx react-native run-ios
```

```
# Build release richiede configurazione signing / Gradle / Xcode
```

Versioning e release notes

Prima della pubblicazione

- Aggiorna versione app e build number.
- Scrivi changelog chiaro (cosa cambia, bug fix, note).
- Mantieni coerenza tra codice, store listing e screenshot.

4) Test e pubblicazione

Store checklist (solo aspetti base)

Tipi di test prima della release

Checklist

- Test funzionali: navigazione, login, API, errori.
- Test UI: layout, testi tagliati, dark mode (se supportata).
- Test performance: liste, immagini, caricamenti lenti.
- Test permessi: camera, notifiche, location (accetta/nega).

Smoke test su device reale

Minimo indispensabile

- Apri app e percorri i flussi principali.
- Simula offline / rete lenta.
- Controlla crash dopo resume/background.
- Verifica notifiche e deep link se presenti.

Debug build vs dev build

Perche e importante

- In dev alcune cose sembrano funzionare grazie a strumenti e overlay.
- La build reale ha comportamento piu vicino all'utente finale.
- Testa sempre una build reale prima di pubblicare.

Google Play - panoramica base

Pubblicazione

- Serve account sviluppatore (non è gratuito).
- Per il corso mostriamo solo il flusso e i materiali richiesti.
- Upload AAB, compila scheda store, privacy, contenuti, test interno/chiuso.

App Store iOS - panoramica base

Pubblicazione

- Serve Apple Developer Program (non gratuito).
- Per il corso: spiegare flusso generale, non per forza pubblicare.
- Serve build iOS firmata, metadati, screenshot, review Apple.

Se vuoi restare 'free' nel corso

Approccio consigliato

- Usa Expo Go e/o dev build per mostrare il funzionamento.
- Fai build interne / file di test senza pubblicare sugli store.

Checklist release (commentata)

Template riutilizzabile

Release checklist

```
// Pre-release checklist (template)
[
  "Versione aggiornata (app + build)",
  "Test API (success + errori + offline)",
  "Test permessi (accetta / nega)",
  "Test Android device reale",
  "Test iOS device reale",
  "Test layout testi lunghi",
  "Build production generata",
  "Store assets pronti (icone, screenshot, descrizioni)"
]
```


Flusso store in parole semplici

- 1) Costruisci la build finale (AAB/iOS build).
- 2) Carichi il file nello store portal.
- 3) Compili pagina store (testi, immagini, privacy).
- 4) Invi in review / test.
- 5) Pubblici quando tutto è approvato.

Esercizio pratico del modulo

Mini app ottimizzata e debuggabile

Obiettivo esercizio

Feed utenti + debug + ottimizzazione

- Schermata lista utenti da API.
- Stati: loading, error, empty, success.
- FlatList ottimizzata.
- Retry e log strutturati.
- Misura tempi e riduci re-render.

Riepilogo finale

Punti chiave

Punti da portare a casa

Sintesi

- Debugging efficace = metodo + strumenti + log chiari.
- Performance = liste giuste, memoization utile, immagini ottimizzate.
- Build reali: con Expo EAS il flusso è più semplice da spiegare e gestire.
- Store publishing: non gratuito, ma il flusso può essere insegnato anche senza pubblicare.

Domande di ripasso

Domanda 1 di 5

1. Quale combinazione aiuta di più a ridurre i problemi di performance nelle liste lunghe?

- ☐ FlatList senza keyExtractor
- ☐ ScrollView + map() di tutti gli elementi
- ☐ Caricare immagini full-size in ogni riga
- ☐ FlatList + keyExtractor stabile + renderItem leggero/memoizzato
- ☐

Domande di ripasso

Domanda 2 di 5

2. Quando conviene usare React.memo in React Native?

- ☐ Solo dopo la pubblicazione sugli store
- ☐ Solo nei componenti che fanno fetch
- ☐ Quando un componente si ri-renderizza spesso con props stabili
- ☐ Sempre, su ogni componente, senza eccezioni

Domande di ripasso

Domanda 3 di 5

3. Con Expo, quale strumento si usa tipicamente per build cloud?

- ☐ CocoaPods
- ☐ Metro Profiler
- ☐ Flipper
- ☐ EAS Build

Domande di ripasso

Domanda 4 di 5

4. Perché testare una build reale oltre alla dev app?

- ☐ Perché Expo Go non mostra il codice
- ☐ Perché in dev non esistono errori
- ☐ Perché il comportamento può cambiare rispetto allo sviluppo
- ☐ Perché la build reale è sempre più veloce al 100%

Domande di ripasso

Domanda 5 di 5

5. Per la pubblicazione sugli store, quale frase è corretta?

- ☐ Google Play e App Store richiedono account sviluppatore a pagamento
- ☐ Entrambi sono gratuiti
- ☐ Solo Android richiede build, iOS no
- ☐ Puoi pubblicare senza metadati